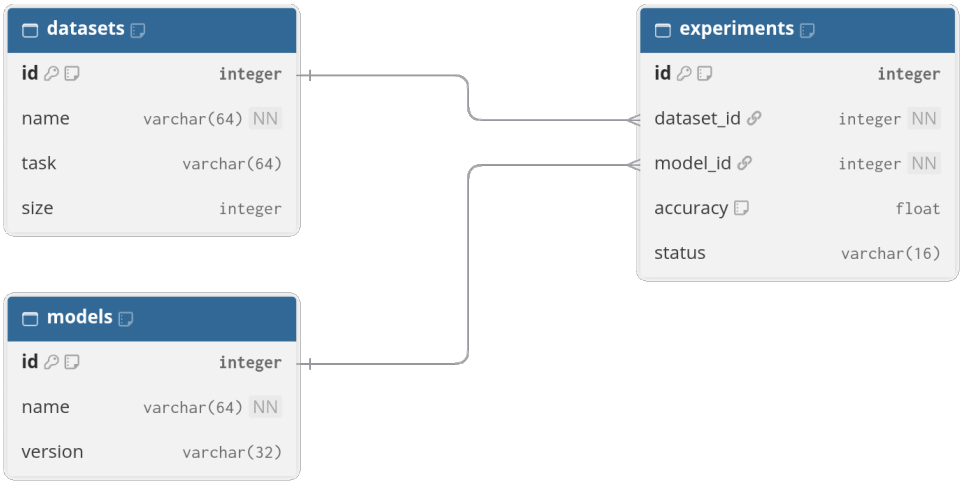
MiniDB · лабораторная работа №1

Схема, устройство и формат хранения

Мини-СУБД реализована на Rust без внешних Rust-зависимостей и готовых СУБД. JSON не используется как основное хранилище. SQL-консоль создаёт, ищет, изменяет и удаляет строки; данные сохраняются после перезапуска.

Три связанные таблицы



datasets(id, name, task, size); models(id, name, version); experiments(id, dataset_id, model_id, accuracy, status). PK — id. FK experiments ссылаются на PK datasets/models. Проверяются уникальность, авто-ID, наличие родителя и RESTRICT.

Собственные файлы

На таблицу: имя_schema — CREATE TABLE, VERSION, GENERATION и счётчик auto-ID; имя_N — единственные текущие данные, N берётся из GENERATION. UTF-8: заголовок, состояния LIVE/DEAD/FREE, значения через Tab. Строки заключены в кавычки, специальные символы экранированы. INT=i64, FLOAT=f64, STRING(n)=UTF-8 до n байт.

ALTER и VACUUM собирают живые строки в новом поколении, синхронизируют данные, атомарно заменяют схему, синхронизируют каталог и удаляют прежний файл. При открытии убираются осиротевшие поколения. ALTER увеличивает VERSION; VACUUM сохраняет структуру и VERSION. Максимум auto-ID сохраняется при удалении мёртвых слотов.

Архитектура

sql.rs → Database (PK/FK, запросы, backup) → Table (один Layout, индекс id → offset, свободные слоты) → TextFile. UTF-8 декодируется в слоты в RAM; обычный CRUD сохраняет атомарный снимок того же файла. SELECT по равенству PK использует индекс; остальные условия просматривают живые строки. Drawio находятся в docs/architecture, PNG — в diagrams.

MiniDB · лабораторная работа №1

Сценарий защиты, проверки и результат индекса

Воспроизведение

Локально: ./sample.sh scratch/defense, затем ./run.sh scratch/defense. Docker: docker compose build; docker compose run --rm minidb sh /usr/local/bin/sample.sh /data/defense; docker compose run --rm minidb minidb /data/defense. Пример содержит 5 datasets, 5 models и 20 experiments; первые три experiments — из ТЗ.

Операции и ожидаемый результат

```
INSERT INTO datasets (name, task, size) VALUES ('demo', 'classification', 43);
SELECT id, name FROM datasets WHERE size >= 100;
UPDATE datasets SET size = 44 WHERE name = 'demo';
DELETE FROM datasets WHERE name = 'demo';
INSERT INTO experiments VALUES (NULL, 999, 1, 0.9, 'running');
```

INSERT выдаёт авто-ID; SELECT выбирает поля с условием; UPDATE и DELETE меняют по одной строке. Последний запрос отклоняется из-за FK. После EXIT и повторного открытия остаются 20 experiments. Также поддерживаются шесть сравнений, AND/OR, NOT, IS NULL, INNER JOIN, ALTER, VACUUM и backup/restore.

Сравнение поиска с индексом и без

Release: 5000 строк, 1000 одинаковых запросов. Наполнение, открытие и sync вне таймера. Данные уже в RAM; это не измерение дискового I/O. Контрольные суммы всех трёх способов совпали: 2486500.

Полный просмотр	66.159 мс
Индекс get(id)	0.143 мс
SELECT WHERE id = …	0.134 мс

Повторить: cargo run --locked --offline --release --example bench -- scratch/bench 5000. Это один локальный замер; результат зависит от машины.

Проверки и ограничения

Автотесты проверяют CRUD, перезапуск, типы, PK/FK, JOIN, фильтры, восстановление поколений и генератор структур. scripts/check-docker.sh повторяет все команды и сценарий защиты в отдельном Docker-томе. Два workflow через act проверяют экспорт drawio в README и генерацию справки из --help.

Docker — инженерное расширение. SQL-парсер, автотесты и генератор — бонусы поверх базовой функциональности. Генератор читает *_schema, потребитель сам компилирует Rust-структуры. WAL и многооператорных транзакций нет; UPDATE ограничен 16 МиБ. Восстановление поколений относится к полной перезаписи. Физическое отключение питания не проверялось.